

Projeto: CRUD_Oficina_Mecanica

Última atualização: 2026-08-28

Resumo

API REST para gerenciamento de uma oficina mecânica com entidades principais: Cliente, Veículo e Ordem de Serviço.

Arquitetura

O projeto segue a arquitetura em camadas:

Controller ↓ Service ↓ Repository ↓ Entity Framework Core ↓ SQL Server

Principais conceitos utilizados:

- DTOs de Request e Response
- Repository Pattern
- Service Layer
- Soft Delete
- Normalização de dados
- ResultadoServico
- Enum com Description

Estrutura do projeto

- Controllers/ - endpoints HTTP (ClienteController, VeiculoController, OrdemServicoController)
- Services/ - regras de negócio e interfaces (IClienteService, IVeiculoService, IOrdemServicoService)
- Repositories/ - acesso a dados (implementações e interfaces)
- Models/ - entidades, DTOs (Requests/Responses), enums
- Validations/ - validadores por entidade
- Datas/ - AppDbContext e configuração do EF Core
- Normalizers/ - normalização de campos (Placa, Telefone, Email)
- Shared/ - classes utilitárias/retorno comum (ex.: ResultadoServico)
- Documents/ - documentação do projeto (este arquivo)

Endpoints implementados

Cliente (api/cliente)

- GET /api/cliente - listar todos os clientes
- GET /api/cliente/{id} - obter cliente por id
- POST /api/cliente - criar cliente
- PUT /api/cliente/{id} - atualizar cliente
- DELETE /api/cliente/{id} - soft delete

Veículo (api/veiculo)

- GET /api/veiculo - listar todos os veículos
- GET /api/veiculo/{id} - obter veículo por id
- POST /api/veiculo - criar veículo
- PUT /api/veiculo/{id} - atualizar veículo
- DELETE /api/veiculo/{id} - soft delete

Ordem de Serviço (api/ordemservico)

- GET /api/ordemservico - listar todas as ordens
- GET /api/ordemservico/{placa} - listar ordens por placa
- POST /api/ordemservico - criar ordem de serviço
- PUT /api/ordemservico/{romaneio} - atualizar ordem de serviço (status, datas, observação)
- DELETE /api/ordemservico/{romaneio} - soft delete

Endpoints / funcionalidades pendentes

- Endpoint GET /api/ordemservico/{romaneio}
- GET /api/ordemservico/{romaneio} - obter ordem por romaneio (implementar se necessário)
- Paginação e filtros para listagens (clientes, veículos, ordens)
- Autenticação e autorização (JWT / Identity)

Regras de Negócio

Cliente

- Exclusão por Soft Delete

Veículo

- Placa normalizada antes de persistir

Ordem de Serviço

- Criação baseada na placa
- Associação automática ao Veiculoid
- Atualização por Romaneio
- Exclusão por Soft Delete
- Status armazenado como enum e exibido como texto

Possíveis Melhorias

- Logging estruturado (Serilog) e correlação de requisições.
- Documentação Swagger mais completa (ex.: examples, responses, versões de API).
- Mapear DTOs com AutoMapper para reduzir código de transformação.
- Cobertura de testes unitários e de integração (xUnit/NUnit) para services e controllers.
- Dockerfile e docker-compose para facilitar execução local/produção.

- Políticas de retry/transações ao interagir com banco de dados quando necessário.
- Migrations do EF Core versionadas e procedimento de deploy seguro.

Checklist

Feito

- Estrutura básica do projeto criada (Controllers, Services, Repositories, Models)
- Endpoints CRUD básicos para Cliente e Veículo implementados
- Endpoints: listar, criar, atualizar e soft delete de Ordem de Serviço implementados
- Swagger configurado (AddSwaggerGen)

Correções realizadas no código (refletidas no repositório):

- Implementado ExistsAsync / removida NotImplementedException residual em repositórios.
- Corrigido typo no DTO de resposta de veículo: Palca → Placa (contratos e mapeamentos atualizados).
- Padronizado método de soft delete: SoftDeleteAsync em repositórios, services e controllers.
- Corrigido typo em nomes de listagem de ordens: ObeterTodos → ObterTodos.
- Removidas ocorrências de NotImplementedException após implementar os métodos necessários.

Faltando / Próximas tarefas

- Implementar/validar endpoints adicionais (GET por romaneio/id se necessário)
- Centralizar tratamento de erros e logging
- Criar/rodar testes automatizados (unitários e integração) focados em: criação, atualização e soft delete de ordens
- Validations: reforçar mensagens e códigos HTTP apropriados

Roadmap

Fase 1

- API REST

Fase 2

- Middleware Global de Exceções
- JWT
- Autorização

Fase 3

- Blazor

Fase 4

- Relatórios

Backlog pós-API (Blazor)

Após a conclusão da API, implementar uma aplicação front-end usando Blazor. Escopo inicial das telas e funcionalidades:

- Tela de Cliente (CRUD): cadastro/edição/remoção (soft delete) de clientes, busca e listagem com paginação e filtros.
- Tela de Funcionários (CRUD): cadastro de funcionários, perfis e associação com permissões.
- Tela de Permissões: gerenciamento de roles/permissões, atribuição a usuários/funcionários.
- Tela de Ordem de Serviço (CRUD): criação de ordens, atribuição a funcionário, alteração de status, histórico e anexos (se necessário).
- Relatórios: geração de relatórios (por período, por funcionário, por veículo/placa, por status) com opção de exportar (PDF/CSV).
- Tela de Login / Autenticação: página de login para acessar o sistema; implementar autenticação (Identity ou JWT) e fluxo de autorização por permissões/roles.

Notas técnicas iniciais:

- Avaliar Blazor Server vs Blazor WebAssembly (hosted) conforme requisitos de escala e tempo de resposta.
- Proteger rotas usando autorização e claims; implementar refresh token se usar JWT.
- Integrar chamadas HTTP à API com HttpClient e tratamento centralizado de erros/timeout.
- Considerar componentes reutilizáveis e design system (ex.: MudBlazor, Radzen ou Bootstrap customizado).

Observação

Observação: este documento deve ser atualizado a cada commit quando houver mudanças relevantes. Para atualizar, edite este arquivo e registre a data e as alterações realizadas.